

Multimedia Application

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: minhaz.ahmed@gmail.com

Content

- Language and grammar
- Regular Expressions
- Words
- Corpora
- Simple Unix Tools for Word Tokenization
- Word Tokenization
- Word Normalization, Lemmatization and Stemming
- Sentence Segmentation

THE BASIC CONCEPTS

- ▶ Languages
- ▶ Grammars

THE BASIC CONCEPTS -Language

▶ LANGUAGES

▶ Any set of string over an alphabet Σ , i.e. any sub set of Σ^* .

▶ Σ (alphabet) : finite nonempty set of symbols.

Ex) Roman alphabet {a, b,...}

Ex) Binary alphabet {0,1}

Ex) Visual object alphabet ??

THE BASIC CONCEPTS -Language

- ▶ String over an alphabet: a finite sequence of symbols from the alphabet.

- ▶ The **length** of a string w : the number of symbols in the string.

$$|uv| = |u| + |v|$$

- ▶ Ex) $w = abaaa$ from $\Sigma = \{a, b\}$: string
→ $|w| = 5$: the length of string w

THE BASIC CONCEPTS -Language

▶ Σ^* :

- ▶ the set of all string obtained by concatenating zero or more symbols including the empty string.

- ▶ Ex) Let $\Sigma = \{a, b\}$.

$$\Sigma^* = \{\lambda, a, b, aa, ab, ba, \dots\} \text{ or } \{a, aa, aab\}$$

$$L = \{a^n b^n : n \geq 0\}$$

▶ λ : empty string

- ▶ $|\lambda| = 0$
- ▶ $\lambda w = w\lambda = w$

▶ $\Sigma^+ : \Sigma^* - \{\lambda\}$

THE BASIC CONCEPTS -Language

- ▶ **Concatenation** of two string w and v : the string obtained the symbol of v to the right end of w .
 - ▶ Ex) Let $w = abc$, $v = def$. Then $wv = abcdef$.
- ▶ w^n : the string obtained by concatenating w with itself n times.
 - ▶ $w^0 = \lambda$.

THE BASIC CONCEPTS -Language

▶ Reverse of a string (w^R):

▶ obtained by writing the symbols in reverse order.

▶ DEFINITION:

1) if w is a string of length 0,

$$w^R = w = \lambda.$$

2) if w is a string of length $n + 1 > 0$, $w = ua$ for some a in Σ ,
then $w^R = a u^R$. $(wx)^R = x^R w^R$

▶ Ex) $w = abc$, $w^R = cba$.

▶ Prefix & suffix: if $w = uv$

▶ The substrings v and u are said to be a **prefix** and a **suffix** of w .

THE BASIC CONCEPTS -Language

- ▶ Concatenation of languages
 - ▶ If L_1 and L_2 are languages over Σ , their concatenation is $L = L_1L_2$,
where $L = \{ w: w = xy \text{ for some } x \text{ in } L_1 \text{ and } y \text{ in } L_2 \}$

Ex) $L_1 = \{aa, ab\}$ and $L_2 = \{ba, bb\}$, then
 $L = \{aaba, aabb, abba, abbb\}$.

THE BASIC CONCEPTS -Language

- ▶ L^* : star closure of a language L
 - ▶ $L^* = L^0 \cup L^1 \cup L^2 \cup \dots$
 - ▶ $L^* = \{ w \text{ in } \Sigma^* : w = w_1 w_2 \dots w_k \text{ for some } k \geq 0 \text{ and some } w_1, w_2, \dots, w_k \text{ in } L \}$
- ▶ L^+ : positive closure of a language L
 - ▶ $L^+ = L^1 \cup L^2 \cup \dots$
- ▶ L^n : L concatenates with itself n times.
 - ▶ Ex) if $L = \{a^n b^n : n \geq 0\}$,
then $L^2 = \{a^n b^n a^m b^m : n \geq 0, m \geq 0\}$

THE BASIC CONCEPTS -Language

► Example 1.10

- If $L = \{a^n b^n : n \geq 0\}$,
then $L^2 = \{a^n b^n a^m b^m : n \geq 0, m \geq 0\}$
- Note that n and m the above are unrelated; the string `aabbbaabbb` is in L^2
- The reverse of L is easily described in set notations as
 $L^R = \{b^n a^n : n \geq 0\}$

THE BASIC CONCEPTS - Grammars

▶ GRAMMARS

- ▶ A typical rule of English grammar is “ a sentence can consist of a noun phrase followed by a predicate.”

<sentence> → <noun_phrase> <predicate>

<noun_phrase> → <article> <noun>

<predicate> → <verb>

- ▶ “a boy runs”, “the dog walks”

THE BASIC CONCEPTS - Grammars

- ▶ GRAMMARS [Definition 1.1]

$$G = (V, T, S, P)$$

- ▶ **V**(variables): a finite set of objects
 - ▶ **T**(Terminal symbols): a finite set of objects (can't change)
 - ▶ **S** $\in V$: a special symbol (start variable)
 - ▶ **P** : a finite set of productions
-
- ▶ V and T are nonempty and disjoint

THE BASIC CONCEPTS - Grammars

- ▶ Production rules:
 - ▶ specify how the grammar transforms one string into another.
- ▶ The form of production rule: $\mathbf{X} \rightarrow \mathbf{y}$
 $\mathbf{x} \in (\mathbf{V} \cup \mathbf{T})^+$ and $\mathbf{y} \in (\mathbf{V} \cup \mathbf{T})^*$

THE BASIC CONCEPTS - Grammars

► Derivation

- $w=uxy$ becomes $z=uyv$ using $x \rightarrow y$.
- $w \Rightarrow z$: w derives z .
- $w_1 \Rightarrow w_2 \Rightarrow \dots \Rightarrow w_n : \mathbf{w_1 \Rightarrow w_n}$
 - unspecified number of steps including zero.
- $w_1 \Rightarrow w_n$: at least one production.
- Sentential form
 - $S \Rightarrow w_1 \Rightarrow w_2 \Rightarrow \dots \Rightarrow w_n \Rightarrow w$
- Sentence

THE BASIC CONCEPTS - Grammars

- ▶ Grammars :
- ▶ Let $G = (V, T, S, P)$ be a grammar.
Then the set $L(G) = \{w \in T^* : S \Rightarrow w\}$
is the language generated by G .

THE BASIC CONCEPTS - Grammars

- ▶ $L(G) = \{w \in T^* : S \xRightarrow{*} w\}$: the language generated by G .
- ▶ Example 1.11
 - ▶ $G = (\{S\}, \{a,b\}, S, P)$ with P given by $S \rightarrow aSb, S \rightarrow \lambda$.
 - ▶ $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$.
 - ▶ $S \Rightarrow aabb$.
 - ▶ Explicit description: $L(G) = \{a^n b^n : n \geq 0\}$.
 - ▶ Cf. $G_1 = (\{S,A\}, \{a,b\}, S, P_1)$
with $S \rightarrow aAb | \lambda, A \rightarrow aAb | \lambda$.

THE BASIC CONCEPTS - Grammars

- ▶ Example 1.12

- ▶ Find a Grammar that generates $L = \{a^n b^{n+1} : n \geq 0\}$

- ▶ Grammar : $G = (\{S, A\}, \{a, b\}, S, P)$

- ▶ Production rule :

- ▶ $S \rightarrow Ab$

- ▶ $A \rightarrow aAb$

- ▶ $A \rightarrow \lambda$

- ▶ Derive a few specific sentences

THE BASIC CONCEPTS - Grammars

- ▶ Equivalent
 - ▶ They generate the same language,
 - ▶ $L(G_1) = L(G_2)$
 - ▶ Two grammars G_1 and G_2 are equivalent.

Natural Language processing

- ▶ NLP is field of computer science and AI that concerned with computer linguistics and interaction between human and computer natural language.
- ▶ Application
 - ❑ Text processing and analysis
 - ❑ Text to speech , speech to text, speech to speech
 - ❑ Machine translation
 - ❑ Search engine.
 - ❑ Sentiment analysis and opinion mining.
 - ❑ Advanced text editor and IDE
 - ❑ Question answering
 - ❑ Spam and fraud detection

Translation

≡ Google Translate

Text

Images

Documents

Websites

Detect language Korean **English** Spanish ▼

how are you?



12 / 5,000  ▼

↔ **Uzbek** Korean English ▼

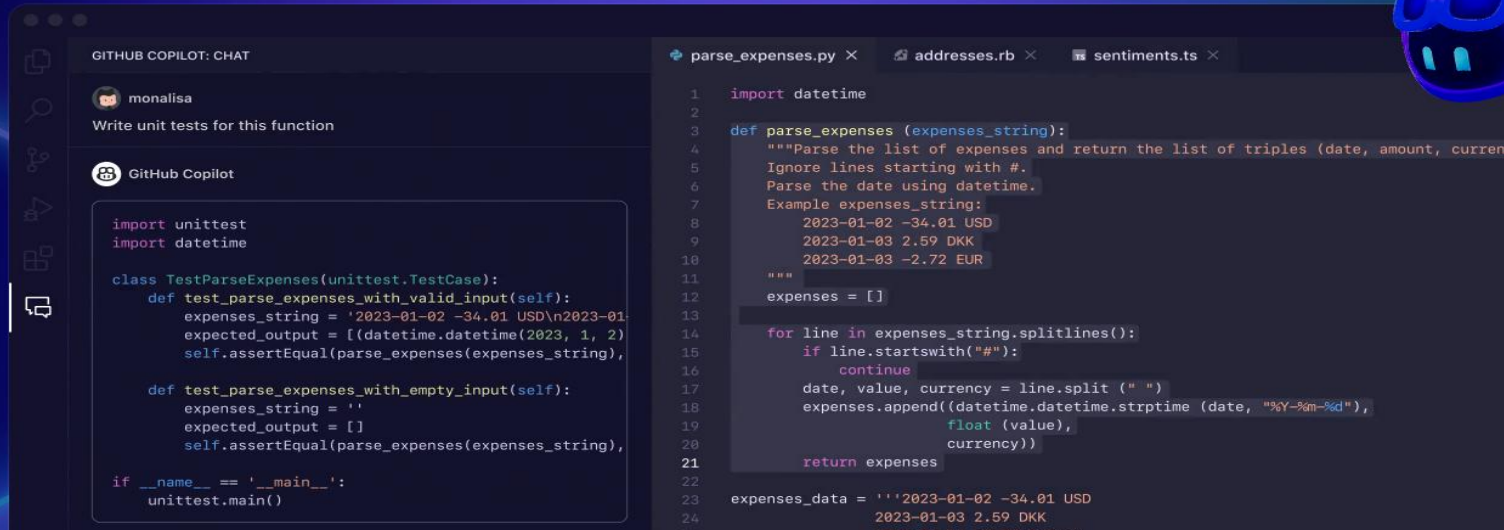
qalaysiz?



<https://github.com/features/copilot/>

The world's most widely adopted AI developer tool.

Get started with Copilot >



The screenshot displays the GitHub Copilot interface within a code editor. On the left, a 'GITHUB COPILOT: CHAT' sidebar shows a conversation with 'monalisa' where the prompt 'Write unit tests for this function' has been entered. The chat response includes Python code for unit testing. The main editor area shows three open files: 'parse_expenses.py', 'addresses.rb', and 'sentiments.ts'. The 'parse_expenses.py' file is active, showing a function 'parse_expenses' that processes a string of expense data into a list of tuples. The code includes comments and a loop that splits the input string by lines and processes each line.

```
import unittest
import datetime

class TestParseExpenses(unittest.TestCase):
    def test_parse_expenses_with_valid_input(self):
        expenses_string = '2023-01-02 -34.01 USD\n2023-01-03 2.59 DKK\n2023-01-03 -2.72 EUR'
        expected_output = [(datetime.datetime(2023, 1, 2), -34.01, 'USD'), (datetime.datetime(2023, 1, 3), 2.59, 'DKK'), (datetime.datetime(2023, 1, 3), -2.72, 'EUR')]
        self.assertEqual(parse_expenses(expenses_string), expected_output)

    def test_parse_expenses_with_empty_input(self):
        expenses_string = ''
        expected_output = []
        self.assertEqual(parse_expenses(expenses_string), expected_output)

if __name__ == '__main__':
    unittest.main()
```

```
1 import datetime
2
3 def parse_expenses (expenses_string):
4     """Parse the list of expenses and return the list of triples (date, amount, currency)
5     Ignore lines starting with #.
6     Parse the date using datetime.
7     Example expenses_string:
8         2023-01-02 -34.01 USD
9         2023-01-03 2.59 DKK
10        2023-01-03 -2.72 EUR
11    """
12    expenses = []
13
14    for line in expenses_string.splitlines():
15        if line.startswith("#"):
16            continue
17        date, value, currency = line.split(" ")
18        expenses.append((datetime.datetime.strptime (date, "%Y-%m-%d"),
19                        float (value),
20                        currency))
21    return expenses
22
23 expenses_data = '''2023-01-02 -34.01 USD
24                 2023-01-03 2.59 DKK
25                 2023-01-03 -2.72 EUR'''
```

NLP Library

- ▶ Natural Language Toolkit (NLTK)
- ▶ Textblob
- ▶ CoreNLP
- ▶ GenSim
- ▶ SpaCy
- ▶ AllenNLP
- ▶ Ployglot

How similar are two strings?

- ▶ Spell correction

- ▶ The user typed “graffe”

Which is closest?

- ▶ graf
 - ▶ graft
 - ▶ grail
 - ▶ giraffe

- Computational Biology

- Align two sequences of nucleotides

```
AGGCTATCACCTGACCTCCAGGCCGATGCCC
TAGCTATCACGACCGCGGTCGATTGCCCGAC
```

- Resulting alignment:

```
-AGGCTATCACCTGACCTCCAGGCCGA--TGCCC---
TAG-CTATCAC--GACCGC--GGTCGATTGCCCGAC
```

- Also for Machine Translation, Information Extraction, Speech Recognition

Regular expression

- ▶ Regular expression (often shortened to regex), a language for specifying text search strings.
- ▶ Application
- ▶ Search in text, pattern in text, string matching, linux terminal
- ▶ E.g. Ctrl+F

Pattern	Example	Description
[0-9]	18524	Match only numeric digit
{A-Z}	AZIZ	Only capital word, If word = 'Aziz' then not matching
[A-Za-z]	Aziz Ahmed	Capital and small letter but not digit such as Aziz98
[Hmm*]	Hm, Hmm, Hmmm, Hmm.	0 or more matching

Regular expression

Regex	Match (single characters)	Example Patterns Matched
/[^A-Z]/	not an upper case letter	“Oy <u>f</u> n pripetchik”
/[^Ss]/	neither ‘S’ nor ‘s’	“ <u>I</u> have no exquisite reason for’t”
/[^.]/	not a period	“ <u>o</u> ur resident Djinn”
/[e^]/	either ‘e’ or ‘^’	“look up <u>^</u> now”
/a^b/	the pattern ‘a^b’	“look up <u>a^b</u> now”

Figure 2.4 The caret ^ for negation or just to mean ^. See below re: the backslash for escaping the period.

Regex	Match	Example Patterns Matched
/woodchucks?/	woodchuck or woodchucks	“ <u>woodchuck</u> ”
/colou?r/	color or colour	“ <u>color</u> ”

Figure 2.5 The question mark ? marks optionality of the previous expression.

Regular expression: Kleene closure

- ▶ The Kleene closure, also known as the Kleene star, is a unary operator denoted by an asterisk (*) symbol. It is used to indicate that the preceding element or subexpression can occur zero or more times.
- ▶ For example, if we have a regular expression "a*", it means that the character 'a' can occur zero or more times. This would match strings such as "", "a", "aa", "aaa", and so on.
- ▶ Similarly, in the regular expression "(ab)*", the Kleene closure applies to the entire subexpression "(ab)", indicating that the sequence "ab" can occur zero or more times. This would match strings such as "", "ab", "abab", "ababab", and so on.

Regular expression: Kleene closure

- ▶ Zero or more character recognition in a text.

- ▶ Application:

Linux terminal

classification :

“0” = zero or more

“1” = 1 or more

Example 1: New(s)^* = New, News, Newss, Newsss, Newsssss

Example 2: New(s)^+ = News, Newsss, Newsssss

Regular expression: Kleene closure

- ▶ Sometimes it's annoying to have to write the regular expression for digits twice, so there is a shorter way to specify “at least one” of some character.
- ▶ One very important special character is the period (`/./`), a **wildcard** expression that matches any single character (*except* a carriage return), as shown in Fig. 2.6.

Regex	Match	Example Matches
<code>/beg.n/</code>	any character between <i>beg</i> and <i>n</i>	<u>begin</u> , <u>beg'n</u> , <u>begun</u>

Figure 2.6 The use of the period `.` to specify any character.

Regular expression: Kleene closure

- ▶ **Anchors** are special characters that anchor regular expressions to particular places in a string. The most common anchors are the caret ^ and the dollar sign \$. The caret ^ matches the start of a line.

Regex	Match
^	start of line
\$	end of line
\b	word boundary
\B	non-word boundary

Figure 2.7 Anchors in regular expressions.

Disjunction, Grouping, and Precedence

- ▶ This idea that one operator may take precedence over another, requiring us to sometimes use parentheses to specify what we mean, is formalized by the **operator precedence hierarchy for regular expressions**.

Parenthesis	()
Counters	* + ? {}
Sequences and anchors	the ^my end\$
Disjunction	

Regular expression: pipe symbol (|)

works similar like logical OR

Choose a particular sub string from the input string

- ▶ Go (es|ing): Goes, Going
- ▶ S (a|u|i) ng: Sing, Sung, Sang
- ▶ Example: Two string

More Operators

Regex	Expansion	Match	First Matches
\d	[0-9]	any digit	Party_of_5
\D	[^0-9]	any non-digit	Blue_moon
\w	[a-zA-Z0-9_]	any alphanumeric/underscore	Daiyu
\W	[^\w]	a non-alphanumeric	!!!!
\s	[\r\t\n\f]	whitespace (space, tab)	in_Concord
\S	[^\s]	Non-whitespace	in_Concord

Figure 2.8 Aliases for common sets of characters.

Regex	Match
*	zero or more occurrences of the previous char or expression
+	one or more occurrences of the previous char or expression
?	zero or one occurrence of the previous char or expression
{n}	exactly <i>n</i> occurrences of the previous char or expression
{n,m}	from <i>n</i> to <i>m</i> occurrences of the previous char or expression
{n,}	at least <i>n</i> occurrences of the previous char or expression
{,m}	up to <i>m</i> occurrences of the previous char or expression

Figure 2.9 Regular expression operators for counting.

Regular expression

Regex	Match	First Patterns Matched
*	an asterisk “*”	“K*A*P*L*A*N”
\.	a period “.”	“Dr. Livingston, I presume”
\?	a question mark	“Why don’t they come and lend a hand?”
\n	a newline	
\t	a tab	

Figure 2.10 Some characters that need to be backslashed.

Words

- ▶ A computer-readable **corpora** collection of text or speech. (plural corpus)
- ▶ For example the Brown corpus is a million-word collection of samples from 500 written English texts from different genres (newspaper, fiction, non-fiction, academic, etc.), assembled at Brown University in 1963–64

Words

Corpus	Instances = N	Types = $ V $
Shakespeare	884 thousand	31 thousand
Brown corpus	1 million	38 thousand
Switchboard telephone conversations	2.4 million	20 thousand
COCA	440 million	2 million
Google n-grams	1 trillion	13 million

Figure 2.11 Rough numbers of wordform types and instances for some English language corpora. The largest, the Google n-grams corpus, contains 13 million types, but this count only includes types appearing 40 or more times, so the true number would be much larger.

Text normalization in NLP

Normalization is the process of converting into standard form.

Necessity of normalization

Word boundary detection

Separated word from each other

Example: Uzbekistan, New York, cats and dogs

Example 2: #nlp

The world has 7097 languages. 2018

Tokenization

- ▶ There are roughly two classes of tokenization algorithms. In [top-down tokenization](#), we define a standard and implement rules to implement that kind of tokenization. In bottom-up tokenization, we use simple statistics of letter sequences to break up words into subword tokens.
- ▶ While the Unix command sequence just removed all the numbers and punctuation, for most NLP applications we'll need to keep these in our tokenization. We often want to break off punctuation as a separate token; commas are a useful piece of information for parsers, periods help indicate sentence boundaries.

Methods of Text Normalization:

► Tokenization

Input: Uzbekistan is beautiful

Output: [Uzbekistan, is, beautiful]

Lemmatization: Finding the same root.

Input: [Sings, sung, sang], [connection, connected, connecting]

Output: [Sing], [connect]

A **lemma** is a set of lexical forms having the same stem, the same major part-of-speech, and the same word sense

Methods of Text Normalization:

- ▶ Stemming: porter stemmer

Rules 1 : ational -> ate, ator -> ate

Example 1: Relational -> Relate, Operator -> Operate

Rule 2: Ing -> Null, s-> Null

Output: Going -> Go, Walking -> Walk, Cats -> Cat

Stemming Challenge : Need to handle some exceptional case

Input: King, Nothing, News

Output: K, noth, New

Methods of Text Normalization:

- ▶ Stop words: Remove the common word that are not important

- ▶ words: am is are, an the

- ▶ Example:

Input: Tashkent has a beautiful park.

Output: Tashkent beautiful park

- ▶ Parts Of Speech Tagging: Identify the grammar of each word

Input: IUT has a beautiful campus

Output: IUT = Noun, has = verb, a = Article, beautiful = adjective , campus = Noun.

Tokenization

- ▶ Tokenization includes word, sentence, whitespace, punctuation.

```
✓ [1] import nltk
```

2s

```
✓ [4] nltk.version_file  
nltk.__author__
```

0s

```
'NLTK Team'
```

```
✓ [5] nltk.download("punkt")
```

0s

```
[nltk_data] Downloading package punkt to /root/nltk_data...  
[nltk_data] Unzipping tokenizers/punkt.zip.  
True
```

Sentence Tokenization

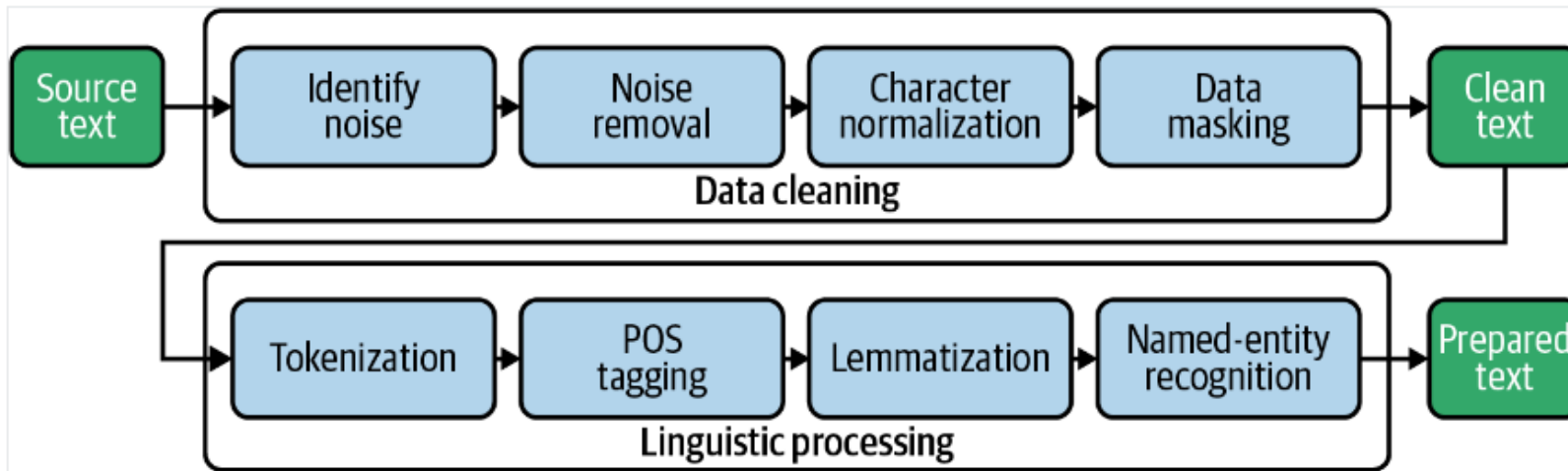
```
✓ [6] from nltk.tokenize import sent_tokenize
```

0s

Sentence Tokenization

Tokenization is a text preprocessing step in sentiment analysis that involves breaking down the text into individual words or tokens.

Preprocessing Text



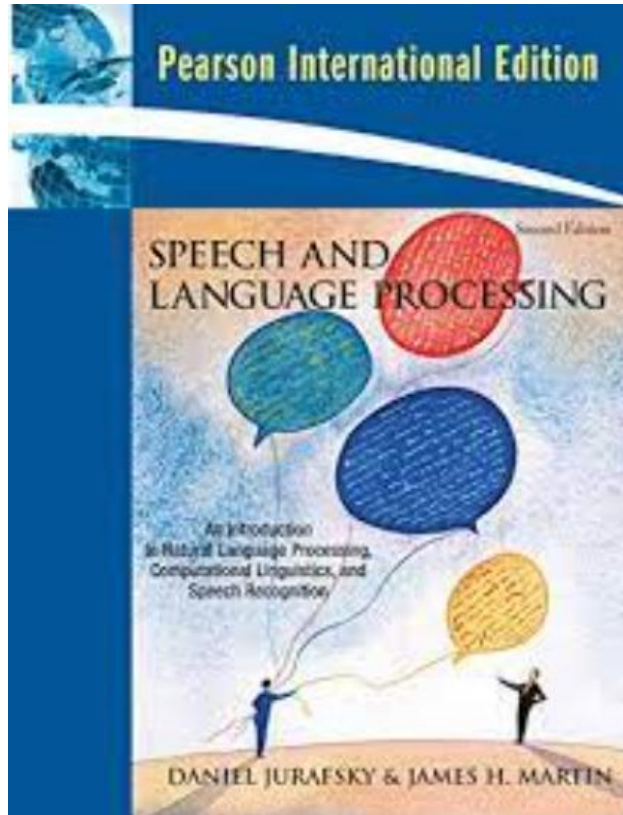
Word tokenization

Word Tokenization

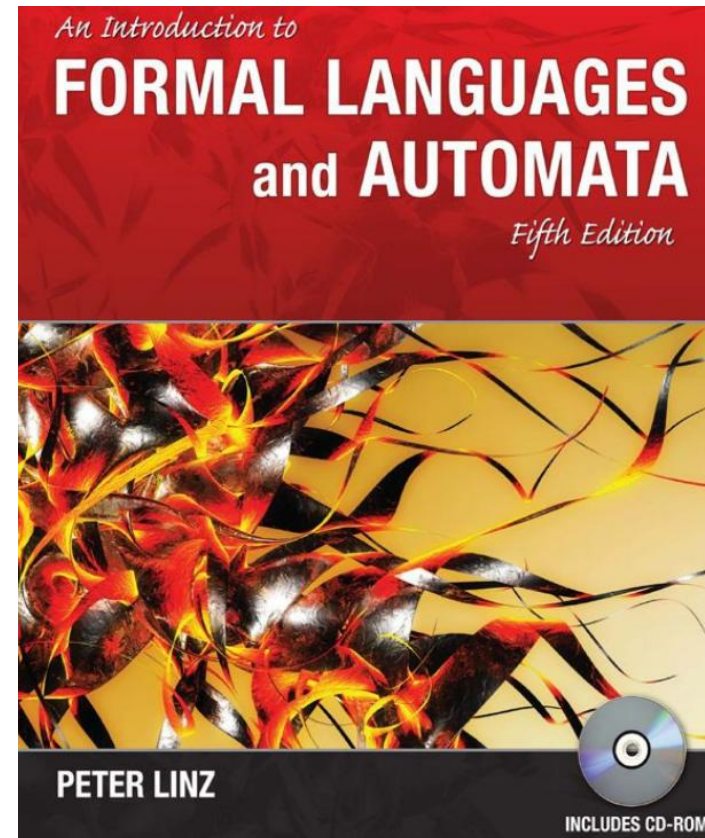
```
✓ 0s  from nltk.tokenize import word_tokenize  
tokenized_word=word_tokenize(text)  
word_tokenize(text)
```

```
['Department',  
'of',  
'Computer',  
'Science',  
'and',  
'Engineering',  
'of',  
'IUT',  
,,  
'started',  
'its',  
'academic',  
'activities',  
'from',  
'2014',  
'with',
```

Reference



Chapter 2



Chapter 1

Question



Thank you

